Basic (your first real API call) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `api_first_call_practice.py`: every later exercise in this document assumes you've already seen, once, that a bad URL and a bad response are two completely different kinds of failure — one never reaches the network, the other comes back from a real server. This script is where you see that split for the first time, with nothing else going on. **If not:** the first time you'd meet that distinction would be buried inside a retry loop or an error-classification function, where it's much harder to isolate what's actually different about the two failure types.

Solution

```
# api_first_call_practice.py
import requests

def main() -> None:
    # why: proves a real call/response round trip works before anything
    # else in this document builds on top of it.
    response = requests.get("https://api.github.com")
    # how: the status code alone tells you the transport succeeded
    print(f"status: {response.status_code}")
    # how: .json() parses the response body — a separate step from the status
    # check
    print(f"body: {response.json()}")

    try:
        # when: a malformed URL fails before any network call even happens —
        # this is a client-side mistake, not a server response.
        requests.get("htp://broken")
    except requests.exceptions.RequestException as e:
        # how: catches only requests' own exception family, not every possible
        # Exception — a bug elsewhere in a bigger try block still surfaces
        # normally.
        print(f"Caught a client-side request error: {type(e).__name__} - {e}")

if __name__ == "__main__":
    main()
```

Expected output:

```
status: 200
body: {'current_user_url': 'https://api.github.com/user', ...}
Caught a client-side request error: MissingSchema - Invalid URL
'htp://broken': No scheme supplied. Perhaps you meant https://htp://broken?
```

(The last line is shown wrapped onto 2 lines just to fit the page — really one line of output.)